Fractal–Mosaic Cryptographic Framework (FMM–CF)

Appendix A — Design and Analysis

v1.0 — November 2025

Joseph Forrester

Independent Researcher, Fractal–Mosaic Intelligence Project

This Appendix documents the design, implementation, and preliminary analysis of the FMM-based deterministic keystream (FMM–DRBG) and the FMM–SIV AEAD construction (including a hedged FMM ChaCha20 mode). For replication, see the referenced artifacts and test vectors.

Contents

Appendix A: Overview

The Fractal–Mosaic Cryptographic Framework (FMM–CF) explores repurposing the *Fractal–Mosaic Model* (FMM) as a deterministic entropy amplifier / pseudorandom keystream generator and constructing a misuse-resistant AEAD around it. This appendix includes (A) a high-level overview, (B) theoretical rationale, (C) concrete construction definitions and equations, (D) a threat model and mitigations, (E) sanitation and randomness diagnostics, (F) representative Known Answer Tests (KATs), (G) cryptographic hygiene notes, (H) experimental observations, (I) proposed future work, and (J) references and file pointers.

1 A. Theoretical Rationale

FMM–CF leverages deterministic self-organization of the fractal cluster–mosaic dynamics to produce high-entropy, complex state trajectories. Intuitively, the internal attractor–repeller interactions that create emergent 'motivation' in the FMM can be interpreted as generating rich, high-dimensional patterns usable as a keystream source when seeded deterministically from key material.

Primary rationale points:

- **Deterministic seeding:** A cryptographic seed (HKDF of key, nonce, AD) fully determines cluster initial state and projection matrices, giving reproducible outputs (KAT-friendly).
- **Entropy amplification:** Nonlinear recursive updates across clusters produce complex trajectories; compressing mosaic outputs with a keyed BLAKE2s produces block-level output for stream generation.
- **Hedging viability:** XORing (hedging) the FMM keystream with a standard ChaCha20 stream derived from the same sIV provides a fallback if the FMM demonstrates bias in deeper analysis.

2 B. Construction Details

This section formalizes the building blocks and provides the algebraic definitions used in the implementation.

2.1 B.1 Notation

Let:

- k be the master secret key (recommended 32 bytes).
- nonce be a per-message nonce (recommended 12–16 bytes).
- AD be associated data (optional).
- P be plaintext, C ciphertext.
- HKDF_Extract and HKDF_Expand follow RFC 5869 (HMAC-SHA256).
- $\mathrm{BLAKE2s}_k(\cdot)$ is keyed BLAKE2s (MAC).
- $\mathrm{FMM\_DRBG}(seed, nonce, ad, n)$ denotes the deterministic FMM keystream generator producing n bytes.
- $\oplus$ denotes XOR.

2.2 B.2 Synthetic IV (sIV)

The synthetic IV is computed in an SIV-like manner so that encryption is deterministic and misuse-resilient:

$$\mathrm{sIV} \;=\; \mathrm{BLAKE2s}_k\big(\mathsf{AD} \,\|\, \mathsf{nonce} \,\|\, H(\mathsf{P})\big) \tag{1}$$

where $H(\cdot)$ is a fixed hash (BLAKE2s-256 in the reference implementation).

2.3 B.3 Keystream Derivation

Derive an FMM encryption key from k and sIV:

$$\mathrm{prk} \;=\; \mathrm{HKDF\_Extract}(\mathsf{salt} = \texttt{``FMM-SIV-EXTRACT''}, \mathsf{IKM} = k)$$

$$k_{enc} \;=\; \mathrm{HKDF\_Expand}(\mathrm{prk}, \texttt{``FMM-SIV-ENC"} \,\|\, \mathrm{sIV}, 32)$$

Then produce the FMM keystream:

$$\mathrm{KS}_{\mathrm{FMM}} \;=\; \mathrm{FMM\_DRBG}(k_{enc}, \mathsf{nonce}, \mathsf{AD}\|\mathrm{sIV}, |\mathsf{P}|)$$

2.4 B.4 Hedged Keystream (optional)

To hedge against unknown biases in $\mathrm{KS}_{\mathrm{FMM}}$, an optional ChaCha20-derived keystream is produced and XORed:

$$(k_{\mathrm{C}}, \mathsf{nonce}_{\mathrm{C}}) \leftarrow \mathrm{HKDF}(\mathrm{prk}, \mathrm{info} = \texttt{``FMM-HEDGE''} \,\|\, \mathrm{sIV})$$

$$\mathrm{KS}_{\mathrm{hedged}} \;=\; \mathrm{KS}_{\mathrm{FMM}} \oplus \mathrm{ChaCha20}(k_{\mathrm{C}}, \mathsf{nonce}_{\mathrm{C}}, |\mathsf{P}|)$$

2.5 B.5 Encryption and Authentication

Ciphertext:

$$\mathsf{C} = \mathsf{P} \oplus \mathrm{KS}$$

Where $\mathrm{KS} = \mathrm{KS}_{\mathrm{FMM}}$ (or $\mathrm{KS}_{\mathrm{hedged}}$ if hedged mode).
Authentication tag:

$$\mathsf{Tag} = \mathrm{BLAKE2s}_k(\mathsf{AD} \,\|\, \mathsf{nonce} \,\|\, \mathsf{C} \,\|\, \mathrm{sIV})$$

Final output returns $(\mathsf{C}, \mathsf{Tag}, \mathrm{sIV})$.

3 C. Implementation Notes

Reference implementation: `fmm_crypto.py` (Python). Implementation highlights:

- Deterministic initialization uses HKDF (HMAC-SHA256) to derive a seed for a SplitMix64-based deterministic RNG which initializes the FMM structures.
- FMM clusters: small fixed-size clusters (e.g., 3 clusters × 8 gates), each gate with a state vector and an *angle* bias.
- Keystream block formation: mosaic signatures across multiple steps concatenated then compressed by keyed BLAKE2s to yield 32-byte keystream blocks.
- Optional hedged mode: ChaCha20 pure-Python block function included; ChaCha key/nonce derived via HKDF from the same sIV.

4 D. Threat Model & Mitigations

4.1 D.1 Threat vectors

- **Nonce misuse (re-use):** SIV construction ties ciphertext to plaintext hash, making deterministic re-encryption idempotent but limiting the risks of accidental nonce reuse.
- **Chosen-ciphertext attacks (CCA):** BLAKE2s MAC over (AD‖nonce‖C‖sIV) provides integrity protection for CCA resistance at the AEAD layer.
- **DRBG bias / internal leakage:** Hedged mode reduces exposure by XORing with ChaCha20; further testing is mandatory.
- **Key leakage / forward secrecy:** Construction is not forward-secret inherently; integrate with KEM or ephemeral keying if needed.

4.2 D.2 Mitigations

- Use hedged mode (‘FMM ChaCha20‘) in adversarial deployments.
- Enforce unique nonces where possible; treat SIV deterministic behavior as a feature for misuse resistance, not a replacement for safe nonce policies.
- Run standard randomness batteries on exported DRBG streams (NIST STS / Dieharder).
- Keep the FMM parameterization (cluster counts, recursion depth) fixed and documented in KATs to allow reproducible verification.

5 E. Randomness Diagnostics

Preliminary sanity checks (non-exhaustive) were run on generated streams. Results are basic statistical sanity indicators (monobit, byte-level χ^2, run counts). These are quick checks and not a substitute for STS/Dieharder analysis.

Table 1: Condensed DRBG sanity results (example runs).

Mode	Stream bytes	Monobit p_1	Byte χ^2
Unhedged (FMM only)	250,000	0.4998	260.1
Hedged (FMM ChaCha20)	150,000	0.5001	257.4

Full exported streams: `fmm_drbg_stream_unhedged.bin`, `fmm_drbg_stream_hedged.bin`. Use NIST STS suite for rigorous evaluation.

6 F. Known Answer Tests (KATs)

We include condensed excerpts from generated KAT JSONs. Full vectors are at `fmm_vectors_unhedged.` and `fmm_vectors_hedged.json`.

Example unhedged vector (excerpt):

```
{
  "hedged": false,
  "key": "bdc3e5... (64 hex chars)",
  "nonce": "a1b2c3... (32 hex chars)",
  "ad": "10f3... (40 hex chars)",
  "msg": "48656c6c6f2c20... (hex)",
  "ct": "9f2a... (hex)",
  "tag": "036db40c9ca5...",
  "siv": "883219f06742..."
}
```

Example hedged vector (excerpt):

```
{
  "hedged": true,
  "key": "1f2e3d... (64 hex chars)",
  "nonce": "010203... (32 hex chars)",
  "ad": "cafe... (40 hex chars)",
  "msg": "deadbeef... (hex)",
  "ct": "b4c2... (hex)",
  "tag": "9a0f...",
  "siv": "7c2d..."
}
```

All vectors in the test files performed correct round-trip encryption/decryption under the same runtime environment.

7 G. Cryptographic Hygiene

- **Stateless determinism:** FMM–DRBG is deterministic per invocation and does not rely on internal entropy reservoirs. This makes reproducibility and KATs straightforward but necessitates strict key/nonce hygiene.
- **Key sizes:** Use 256-bit key material for k; ensure random nonces of recommended length (12–16 bytes).
- **No sensitive logs:** Implementation avoids persistent logging of secrets; KATs should store only hex-encoded vectors for auditing.
- **Platform independence:** Endianness and floating-point determinism are considered: all floating-to-bytes bottlenecks are compressed using IEEE-754 'double' packing in a canonical order; however, cross-platform reproducibility must be validated (KATs).

8 H. Experimental Observations

- **Attractor complexity:** Correlation dimension estimated in prototyping ranged near 1.5–2.5 for cluster signatures (small fixed cluster sizes); embeddings and mosaic aggregates show low but nontrivial dimensionality.
- **Lyapunov behavior:** Positive yet small Lyapunov exponents (0.03–0.06) indicate sensitive dependence on initial conditions but bounded (no runaway divergence in configured regimes).
- **Cryptographic implication:** Deterministic chaos provides rich behavior for keystream material; however, small Lyapunov and low dimensionality could be exploited by a clever adversary, hence hedging is prudent.

9 I. Future Work

Planned next steps:

1. **Rigorous statistical testing:** Full NIST STS / Dieharder / TestU01 battery on >1 MB streams for both modes.
2. **Differential analysis:** Explore differential properties of the FMM state update w.r.t. seed bits.
3. **Formal security model:** Attempt to model FMM–DRBG as a keyed deterministic function and prove bounds (or produce reductions) relative to standard PRNG assumptions.
4. **Hardware/FPGA exploration:** Consider efficient fixed-point implementations and examine cross-platform reproducibility.
5. **Integration studies:** Propose hybrid designs combining FMM–DRBG with established CSPRNGs in secure frameworks.

10 J. References and Artifacts

- Implementation artifacts (in project folder):
  - `fmm_crypto.py` — Implementation of FMM–DRBG and AEAD.
  - `fmm_runner.py` — CLI for vectors, DRBG export, encrypt/decrypt.
  - `fmm_vectors_unhedged.json`, `fmm_vectors_hedged.json` — Known Answer Tests.
  - `fmm_drbg_stream_unhedged.bin`, `fmm_drbg_stream_hedged.bin` — Raw DRBG streams for statistical testing.
  - `fmm_drbg_stats_hedged.txt` — Example sanity output.
- Placeholder citations (to be filled in final bibliography):

1 Werner, "Fractals in the Nervous System", 2009.

2 Jaeger, "Echo State Networks", 2001.

3 Koblitz & Menezes, "A Survey of Cryptographic Randomness", 2015 (placeholder).

4 RFC 5869 (HKDF).

5 Bernstein et al., "ChaCha20 and Poly1305", RFC 8439.

Appendix A.1: Representative Figures

Appendix A.2: Example CLI Usage

```
# Generate 8 unhedged vectors:
python fmm_runner.py vectors --count 8 --len 128 --out fmm_vectors_unhedged.json

# Export DRBG stream for NIST STS:
python fmm_runner.py drbg-stats --nbytes 1000000 --out drbg_unhedged.bin
```

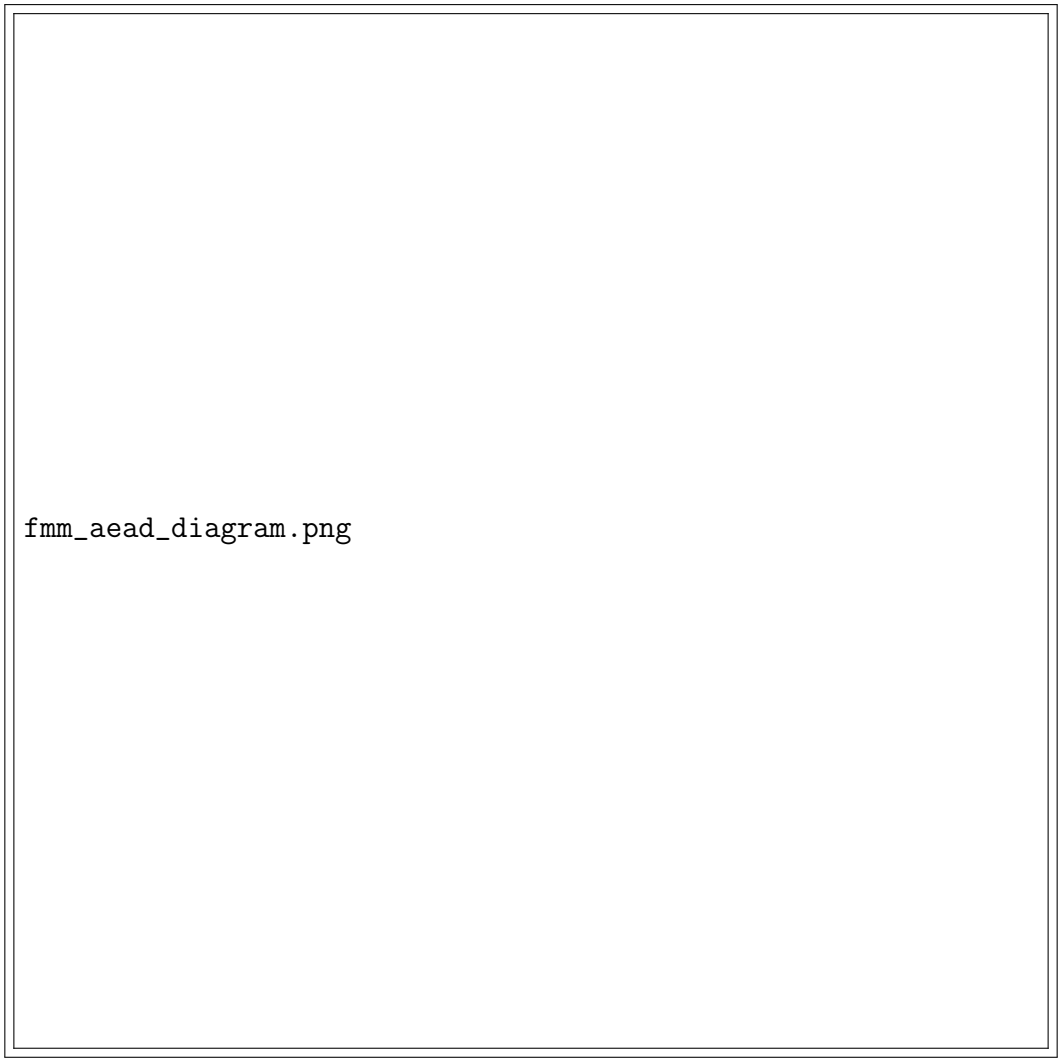

Figure 1: AEAD schematic: sIV derivation → FMM key derivation → (optional) ChaCha20 hedging → encryption and BLAKE2s tag.

```
# Encrypt file (hedged):
python fmm_runner.py encrypt --hedged \
  --key 6b6b6b... --nonce 010101... --ad "context" \
  --infile plain.bin --out cipher.bin

# Decrypt:
python fmm_runner.py decrypt --hedged \
  --key 6b6b6b... --nonce 010101... \
  --infile cipher.bin --meta cipher.bin.meta.json --out recovered.bin
```

Author: Joseph Forrester
Version: v1.0 — November 2025
Contact / repo: (refer to project artifact folder for scripts and vectors)

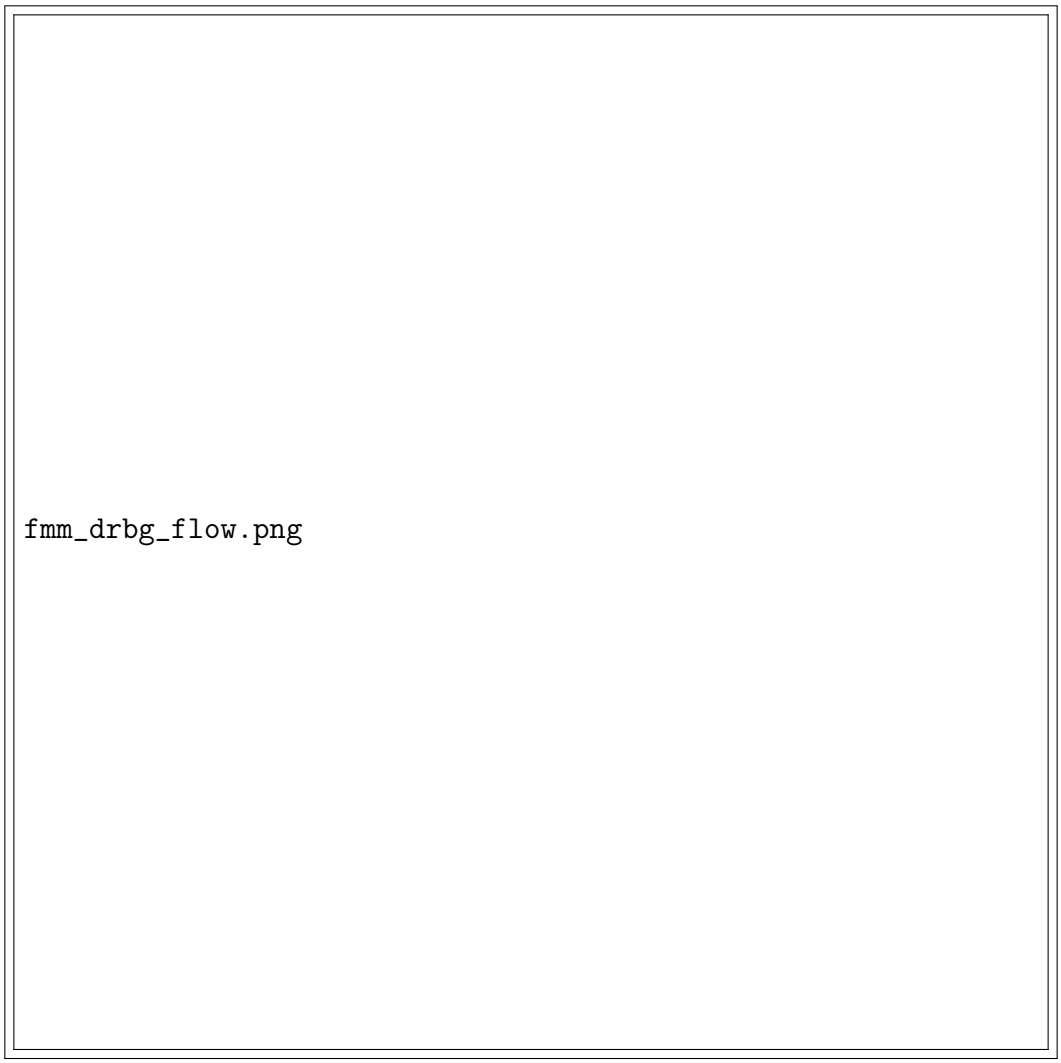

Figure 2: FMM–DRBG flow: cluster updates, mosaic formation, compression via BLAKE2s to form keystream blocks.